# Hackathon Projects

- Eight starter notebooks, each a small experiment that already runs
- Every one pairs a quantum circuit with a classical model
- Pick one this afternoon and make it yours for November 13

# How every notebook works

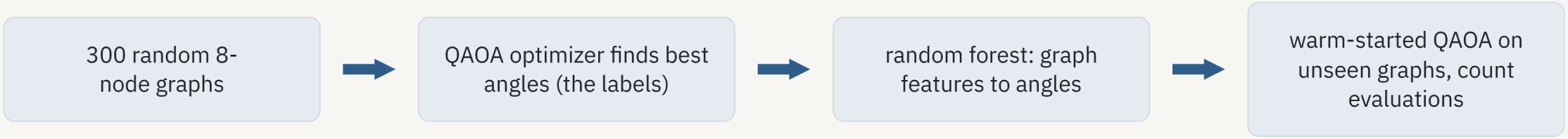
- Run it top to bottom once. The base run takes one to two minutes and saves a figure and a pkl.
- Every constant lives in one config cell at the top. Change a number there, rerun, compare.
- The bottom lists shred-outs: each one is the question your poster answers and the number that answers it




None of them assumes the quantum piece wins. The poster reports what happened, including a tie or a loss.

# 1. Can a model predict good QAOA starting angles?

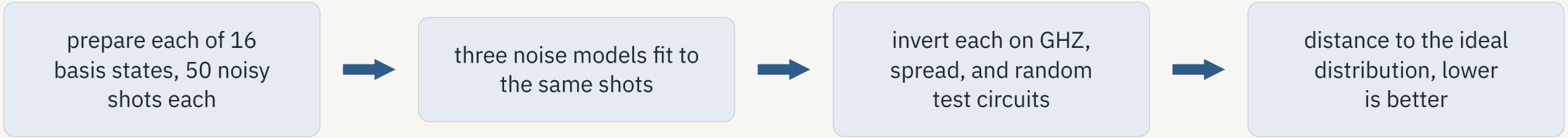
- QAOA solves Max-Cut by tuning two circuit angles. Every new graph starts the optimizer from scratch.
- A random forest reads cheap graph features (edges, degrees, triangles) and predicts the best angles found on training graphs
- The number: optimizer evaluations to reach 98% of the best cut, warm start versus random start, on graph shapes the model never saw




hackaroo26-1-qaoa-warm-start. Base run one to two minutes. Seven shred-outs, from better features to shots instead of exact statevectors.

# 2. Which model of readout noise mitigates best on a small budget?

- Every measurement misreads 0 as 1 and 1 as 0 some fraction of the time. The standard fix inverts a confusion table with $2^n$ columns.
- Three models of the same noise from the same calibration shots: the raw table (240 parameters), logistic regression (75), independent per-qubit (8)
- The number: total variation distance between the mitigated and the ideal distribution on three test circuits




hackaroo26-2-readout-classifier. Base run under a minute. Six shred-outs, from correlated noise to mitigating one observable instead of a distribution.

# 3. Does a quantum kernel classify real data as well as RBF?

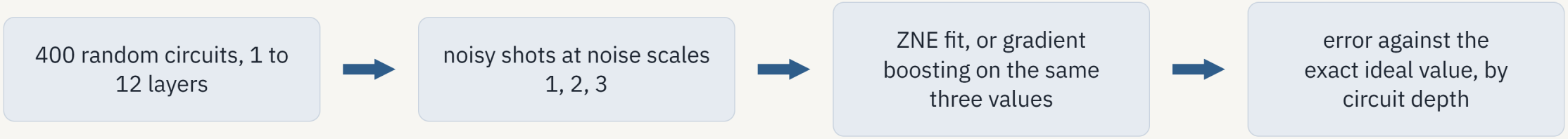
- A feature-map circuit turns a data row into a quantum state. The overlap between two such states is a kernel, and a support vector machine needs nothing else.
- The same SVM on the quantum kernel, the RBF kernel, and the linear kernel, on the same split of the breast cancer dataset reduced to four features
- The number: test accuracy, scored once, after the entangling scale is chosen on a validation set




hackaroo26-3-quantum-kernel-svm. Base run a few seconds. Six shred-outs, from feature map design to kernel concentration versus qubit count.

# 4. Can a regressor beat zero-noise extrapolation?

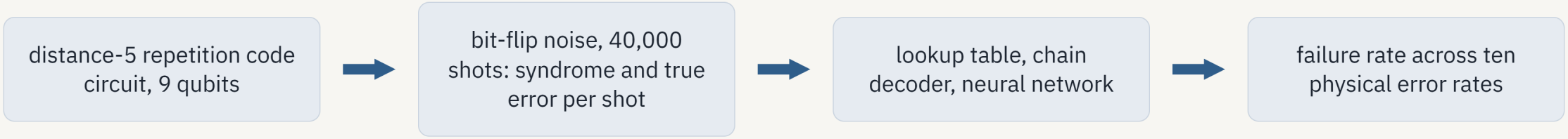
- Gate noise pulls measured expectation values away from their ideal values. ZNE runs the circuit at raised noise and extrapolates back to zero, which amplifies shot noise.
- Gradient boosting sees the same three noisy values plus cheap circuit features and predicts the ideal value, trained on circuits with known answers
- The number: mean absolute error against the ideal on unseen circuits, with an unmitigated 3x-shots baseline at the same total budget




hackaroo26-4-learned-error-mitigation. Base run about a minute. Seven shred-outs, from fewer scales to gate folding and richer noise.

# 5. Can a neural network decode an error-correcting code?

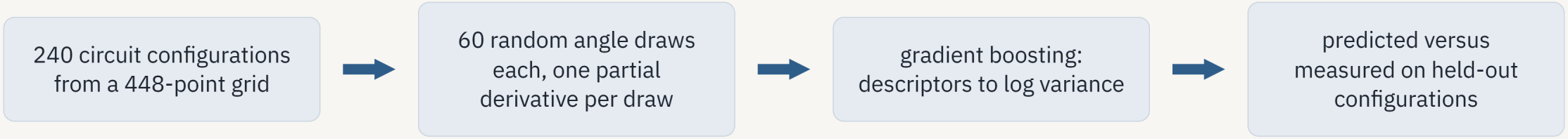
- A code does not fix errors by itself. It produces a syndrome, a few bits that say where neighbors disagree, and a decoder guesses the error from it.
- Two exact baselines (a lookup table and a chain walk) are optimal here. A small PyTorch network is trained on 40,000 simulated shots and must at least tie them.
- The number: recovery failure rate against physical error rate on a log-log plot, with the unencoded line and the exact curve




hackaroo26-5-repetition-code-decoder. Base run under a minute. Seven shred-outs; noisy measurement, correlated noise, and circuit-level noise are where learning can win.

# 6. Can a model predict when a circuit’s gradient vanishes?

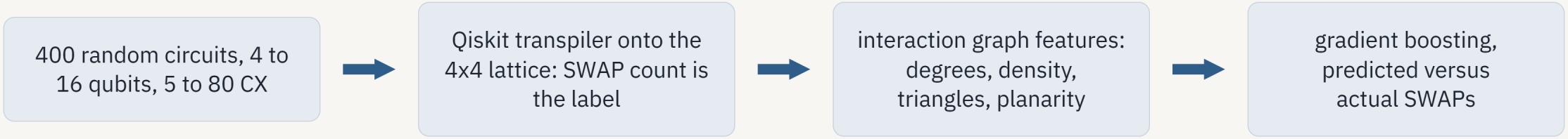
- Variational circuits are trained by gradient descent on their angles. For many circuit families the gradient variance shrinks exponentially with qubit count, and the optimizer has nothing to follow.
- Measure the variance by the parameter-shift rule across a grid of qubit counts, depths, entangling patterns, and cost functions. Gradient boosting predicts log variance from eight descriptors.
- The numbers: variance lost per added qubit in a controlled sweep, and mean absolute error in $\log_{10}$ variance on held-out configurations




hackaroo26-6-barren-plateau-predictor. Base run one to two minutes. Seven shred-outs, from death depth to extrapolating to sizes you cannot simulate.

# 7. Can a model predict routing cost before compiling?

- Hardware lets only neighboring qubits interact. The compiler inserts SWAP gates to move qubits around, and each SWAP is three CX gates of extra noise.
- Random circuits are compiled onto a 4x4 lattice. Gradient boosting predicts the SWAP count from features of the circuit’s interaction graph, against the same model on CX count alone.
- The number: mean absolute error in SWAP count on held-out circuits, graph features versus CX count only




hackaroo26-7-compilation-cost-predictor. Base run under a minute. Eight shred-outs, from compiler seeds to searching the initial layout yourself.

# 8. Do circuit readout statistics carry graph structure that classical features miss?

- One qubit per node, one entangling gate per edge, no training. Statistics of the measurement distribution are the graph’s vector.
- Three graph families with the same node count and edge count. The same classifier on the quantum embedding, each of its three blocks, classical features, and both together.
- The number: balanced accuracy on graph shapes never seen in training. Chance is one third.




hackaroo26-8-quantum-graph-embedding. Base run under a minute. Eight shred-outs, from changing the task to hard pairs that fool the classical features.

# Picking a project

| Notebook | Quantum piece | Classical model | Extra libraries | Base run |
|---|---|---|---|---|
| 1 QAOA warm start | QAOA circuits, exact statevector | random forest | networkx | 1 to 2 min |
| 2 readout classifier | noisy readout simulation | logistic regression | qiskit-aer | under 1 min |
| 3 quantum kernel SVM | feature-map state overlaps | support vector machine | none | seconds |
| 4 learned error mitigation | depolarizing noise, ZNE | gradient boosting | qiskit-aer | about 1 min |
| 5 repetition-code decoder | error-correction circuit, bit-flip noise | PyTorch network | qiskit-aer, torch | under 1 min |
| 6 barren plateau predictor | parameter-shift gradients | gradient boosting | none | 1 to 2 min |
| 7 compilation cost predictor | Qiskit transpiler, 4x4 lattice | gradient boosting | networkx | under 1 min |
| 8 quantum graph embedding | encoding circuit statistics | gradient boosting | networkx | under 1 min |

1:30 run your starter. 3:15 make it yours. Luke is floating.